Supervised Learning - Part 2

100 Practice Questions: Decision Trees, KNN, SVM, Neural Networks & Autoencoders

GARP RAI Exam Preparation

Q1: Decision Trees - Definition

A decision tree is best described as:

1. A supervised learning technique that uses a tree-like structure to make predictions by sequentially splitting data based on feature values.
2. An unsupervised learning technique used to find natural groupings in data.
3. A reinforcement learning algorithm that learns through trial and error.
4. A probabilistic graphical model that represents conditional dependencies.

Answer: A Explanation: A decision tree is a supervised machine learning technique that makes predictions using a tree-like structure with root, branches, internal nodes, and leaves. Each internal node tests a feature, each branch represents an outcome, and each leaf represents a class label (classification) or continuous value (regression). This sequential splitting process makes decision trees highly interpretable "white-box" models, in contrast to "black-box" models like neural networks.

Q2: CART Acronym

What does CART stand for in the context of decision trees?

1. Classification and Regression Trees
2. Cluster Analysis and Regression Trees
3. Classification and Random Trees
4. Correlation and Regression Trees

Answer: A Explanation: CART stands for Classification and Regression Trees, highlighting that decision trees can handle both categorical target variables (classification) and continuous target variables (regression). This dual capability makes them versatile tools in machine learning. The term emphasizes that the same fundamental algorithm can be applied to both types of problems, with different splitting criteria used depending on the nature of the target variable.

Q3: Decision Tree Interpretability

Why are decision trees often called "white-box models"?

1. Because they use white-box algorithms exclusively.
2. Because their visual, flowchart-like structure makes the decision-making process easy to understand and interpret.
3. Because they only work with white data.
4. Because they require white-box hardware.

Answer: B Explanation: Decision trees are called "white-box models" because their logical structure is highly transparent and interpretable. Unlike "black-box" models such as neural networks where internal workings are difficult to decipher, decision trees visually display the entire decision path. Each split, condition, and resulting prediction can be traced and understood, making them particularly valuable in risk management and regulated industries where model interpretability is essential.

Q4: Decision Tree Components

In a decision tree, what does each internal node represent?

1. A class label or prediction value.
2. A "test" on a feature (e.g., "Is pressure ¿ 100 psi?").
3. The outcome of a test (e.g., "Yes" or "No").
4. The root of the tree.

Answer: B Explanation: Each internal node in a decision tree represents a test on a feature. For example, "Is the machine's pressure greater than 100 psi?" or "Is credit score ¿ 700?". Each branch from that node represents the outcome of the test (e.g., "Yes" or "No"), and each leaf node represents a class label (in classification) or a continuous value (in regression). The root node is the topmost internal node where the first split occurs.

Q5: Regression Trees - Target Variable

When are decision trees called regression trees?

1. When the target variable is categorical.
2. When the target variable is continuous.
3. When the target variable is binary.
4. When there are no target variables.

Answer: B Explanation: Decision trees are called regression trees when the target variable is continuous (numerical). The goal is to predict a numerical value such as house price, stock return, or temperature. When the target variable is categorical (e.g., "Fraud" vs "Not Fraud"), they are called classification trees. Both types use the same underlying tree structure but differ in their splitting criteria and prediction methods.

Q6: Regression Tree Prediction

How does a regression tree make a prediction for a new observation?

1. By taking the median of all observations in the leaf node.
2. By taking the average of the training observations in the leaf node where the observation falls.
3. By taking the mode of the training observations in the leaf node.
4. By randomly selecting a value from the leaf node.

Answer: B Explanation: For regression trees, the prediction for any new observation is simply the average (mean) of the training observations in the leaf node where the observation falls. The leaf node represents a region of the feature space that has been partitioned during tree construction. This average value minimizes the residual sum of squares (RSS) within that region, making it the optimal prediction under the tree's structure.

Q7: Regression Tree Splitting Criterion

What is the primary criterion used to determine splits in a regression tree?

1. Maximizing the Gini coefficient.
2. Minimizing entropy.
3. Minimizing the Residual Sum of Squares (RSS).
4. Maximizing information gain.

Answer: C Explanation: Regression trees minimize the Residual Sum of Squares (RSS) to determine splits. The RSS measures the total squared difference between observed target values and the predicted mean value within each region. By minimizing RSS, the tree creates regions that are as homogeneous as possible with respect to the target variable. This is in contrast to classification trees which use entropy or Gini coefficient to measure impurity.

Q8: RSS Formula

What is the formula for Residual Sum of Squares (RSS) in regression trees?

1. $RSS = \sum_{j=1}^{J} \sum_{i \in R_j} (y_i - \bar{y}_{R_j})^2$
2. $RSS = \sum_{j=1}^{J} \sum_{i \in R_j} |y_i - \bar{y}_{R_j}|$
3. $RSS = \sum_{j=1}^{J} \sum_{i \in R_j} (y_i - \bar{y}_{R_j})$
4. $RSS = \sum_{j=1}^{J} \sum_{i \in R_j} (y_i - \bar{y}_{R_j})^3$

Answer: A Explanation: The RSS formula is $RSS = \sum_{j=1}^{J} \sum_{i \in R_j} (y_i - \bar{y}_{R_j})^2$ where y_i is an observation in the training set, $\bar{y}_{R_j}$ is the average outcome of observations in region (leaf) j, and J is the total number of regions. This measures the squared differences between actual values and the region mean. Minimizing RSS ensures that each region is as homogeneous as possible.

Q9: Recursive Binary Splitting

What is recursive binary splitting in the context of decision trees?

1. A bottom-up approach that builds the tree from leaves to root.
2. A top-down approach that sequentially splits the data to minimize RSS or impurity.
3. A random approach that selects splits randomly.
4. A method that splits data into multiple branches at once.

Answer: B Explanation: Recursive binary splitting is a pragmatic "top-down" approach to building decision trees. It starts with all observations in a single region (root), then searches through every feature and every possible split point to find the single split that maximizes RSS reduction (for regression) or impurity reduction (for classification). The process repeats independently on each resulting region until a stopping criterion is met. This approach is computationally feasible compared to trying every possible partition.

Q10: Stopping Criteria

Which of the following is NOT a typical stopping criterion for growing a decision tree?

1. Minimum number of observations in a node.
2. Maximum depth of the tree.
3. Maximum number of leaves.
4. Minimum RSS value reached.

Answer: D Explanation: Typical stopping criteria include: (1) minimum number of observations in a node (e.g., no further splits if node has ¡ 5 observations), (2) maximum depth of the tree (e.g., stop at depth 10), and (3) maximum number of leaves. "Minimum RSS value reached" is NOT a typical stopping criterion because RSS will continue to decrease with more splits; the goal is to prevent overfitting, not achieve a specific RSS value. Stopping criteria are designed to balance model complexity with generalization.

Q11: Classification Trees - Target Variable

When are decision trees called classification trees?

1. When the target variable is continuous.
2. When the target variable is categorical.
3. When the target variable is binary only.
4. When there are no features.

Answer: B Explanation: Decision trees are called classification trees when the target variable is categorical (e.g., "Fraud" vs "Not Fraud", "Yes" vs "No", or multi-class like "Red", "Green", "Blue"). The goal is to assign observations to a specific class. While binary classification is common, classification trees can handle multiple classes. The splitting criterion differs from regression trees, focusing on node purity rather than RSS minimization.

Q12: Node Purity in Classification Trees

What is the goal of splitting in classification trees?

1. To maximize the number of nodes.
2. To create nodes that are as "pure" as possible (ideally all observations in one class).
3. To minimize the number of leaves.
4. To maximize the depth of the tree.

Answer: B Explanation: The objective of splitting in classification trees is to create nodes that are as pure as possible - meaning that the observations within each node ideally all belong to a single class. A perfect split would create nodes containing only one class. Since perfect purity is rare, impurity measures (like entropy and Gini coefficient) are used to quantify node purity and guide splits toward maximizing purity (minimizing impurity).

Q13: Entropy Definition

What is entropy in the context of classification trees?

1. A measure of order in a system.
2. A measure of disorder or impurity in a node.
3. A measure of the number of features.
4. A measure of tree depth.

Answer: B Explanation: Entropy is a measure of disorder or impurity in a system. In classification trees, entropy measures the impurity of a node. A node with all observations from the same class has entropy $= 0$ (perfect purity). A node with observations equally split among classes has maximum entropy. The formula is $Entropy = -\sum_{j=1}^{J} p_j \log_2(p_j)$ where p_j is the probability of the j-th outcome. Lower entropy indicates higher purity.

Q14: Entropy Formula

What is the correct formula for entropy?

1. $Entropy = -\sum_{j=1}^{J} p_j \log_2(p_j)$
2. $Entropy = \sum_{j=1}^{J} p_j \log_2(p_j)$
3. $Entropy = 1 - \sum_{j=1}^{J} p_j^2$
4. $Entropy = -\sum_{j=1}^{J} p_j \log_e(p_j)$

Answer: A Explanation: The entropy formula is $Entropy = -\sum_{j=1}^{J} p_j \log_2(p_j)$ where J is the total number of possible outcomes/unique classes and p_j is the probability of the j-th outcome. The base-2 logarithm is used so entropy can be interpreted as the expected number of bits needed to store the information. Changing the logarithm base only scales the measure by a constant and doesn't affect the relative comparisons between splits.

Q15: Gini Coefficient Definition

What is the Gini coefficient in the context of classification trees?

1. A measure of node impurity.
2. A measure of tree depth.
3. A measure of feature importance.
4. A measure of model accuracy.

Answer: A Explanation: The Gini coefficient (or Gini index) is a measure of impurity in classification trees. It calculates the probability of misclassifying a randomly chosen element if it were randomly labeled according to the class distribution in the node. The formula is $Gini = 1 - \sum_{j=1}^{J} p_j^2$. A small Gini value indicates a node mostly contains instances from the same class (high purity), while a large value indicates high impurity.

Q16: Gini Coefficient Formula

What is the correct formula for the Gini coefficient?

1. $Gini = 1 - \sum_{j=1}^{J} p_j^2$
2. $Gini = \sum_{j=1}^{J} p_j^2$
3. $Gini = -\sum_{j=1}^{J} p_j \log_2(p_j)$
4. $Gini = \sum_{j=1}^{J} p_j \log_2(p_j)$

Answer: A Explanation: The Gini coefficient formula is $Gini = 1 - \sum_{j=1}^{J} p_j^2$ where J is the number of classes and p_j is the proportion of observations in class j within the node. When a node is perfectly pure (all observations in one class), $p_j = 1$ for one class and $p_j = 0$ for all others, so $Gini = 1 - 1^2 = 0$. When a node has equal proportions across classes, the Gini coefficient is maximized.

Q17: Entropy vs Gini Comparison

How do entropy and Gini coefficient compare in practice for classification trees?

1. Entropy always produces better trees than Gini.
2. Gini always produces better trees than entropy.
3. They usually lead to very similar decision trees in practice.
4. They cannot be used together in the same tree.

Answer: C Explanation: In practice, entropy and Gini coefficient usually lead to very similar decision trees. Both measures aim to quantify node impurity and guide splits toward higher purity. While they are derived from different theoretical foundations (entropy from information theory, Gini from probability), their practical performance is comparable. The choice between them is often a matter of preference or computational efficiency, with Gini being slightly faster to compute.

Q18: Information Gain Definition

What is information gain in the context of decision trees?

1. The reduction in impurity (entropy or Gini) achieved by a split.
2. The increase in tree depth.
3. The number of features used in the tree.
4. The accuracy of the tree on training data.

Answer: A Explanation: Information gain is the reduction in impurity achieved by a split. It is calculated as the difference between the impurity of the parent node and the weighted average impurity of the child nodes. For example, if the parent node has Gini $= 0.480$ and the weighted average Gini of child nodes is 0.255, the information gain is 0.225. The feature and split point that maximize information gain are selected for each node.

Q19: Root Node Selection

How is the root node feature selected in a classification tree?

1. Randomly selected from all features.
2. The feature with the highest information gain (largest reduction in impurity) is selected.
3. The feature with the lowest information gain is selected.
4. All features are used at the root node.

Answer: B Explanation: The root node feature is selected by evaluating all features and choosing the one that provides the highest information gain (largest reduction in impurity). For categorical features, each category is considered. For continuous features, an iterative procedure searches for the optimal threshold that maximizes information gain. The feature that yields the purest split becomes the root node, as it provides the most informative initial partition of the data.

Q20: Continuous Feature Splitting

How are continuous features handled in decision trees?

1. They are converted to categorical features first.
2. They are ignored in decision trees.
3. An iterative procedure selects the optimal threshold that maximizes information gain.
4. They are used only in regression trees, not classification trees.

Answer: C Explanation: For continuous features, decision trees use an iterative procedure to find the optimal threshold that maximizes information gain (or minimizes impurity). The algorithm tests various split points (e.g., thresholds like "Retail_investor ¿ 35%") and selects the one that results in the greatest reduction in impurity. This is typically an automated process in machine learning software and is essential for handling numerical features effectively.

Q21: Pruning - Definition

What is pruning in the context of decision trees?

1. Adding more branches to the tree to increase complexity.
2. Reducing the size of the tree to avoid overfitting and enhance interpretability.
3. Removing features from the dataset.
4. Increasing the depth of the tree.

Answer: B Explanation: Pruning is a technique to reduce the size of a decision tree by removing branches or subtrees that do not contribute significantly to predictive accuracy. The goal is to avoid overfitting and enhance interpretability. Pruning can be performed pre-pruning (stopping tree growth early) or post-pruning (growing the tree fully then removing weak links). Smaller trees are generally more interpretable and generalize better to new data.

Q22: Pre-Pruning vs Post-Pruning

What is the difference between pre-pruning and post-pruning?

1. Pre-pruning is done after tree growth; post-pruning is done during tree growth.
2. Pre-pruning is done during tree growth; post-pruning is done after the tree is fully grown.
3. Pre-pruning uses Gini; post-pruning uses entropy.
4. There is no difference; they are the same.

Answer: B Explanation: Pre-pruning (also called online pruning) occurs during tree growth by imposing stopping criteria (e.g., minimum number of observations in a node, maximum depth, maximum number of branches). Post-pruning (also called offline pruning) involves growing the tree fully and then removing weak links by replacing subtrees with leaves. Post-pruning algorithms include reduced error pruning (bottom-up) and cost-complexity pruning.

Q23: Reduced Error Pruning

What is reduced error pruning?

1. A top-down pruning approach that starts at the root.
2. A bottom-up approach that replaces nodes with the most popular class if the pruned tree doesn't perform worse on validation data.
3. A method that uses entropy to determine which nodes to prune.
4. A method that increases tree complexity.

Answer: B Explanation: Reduced error pruning is a bottom-up approach where, starting from the leaves, each node is replaced with its most popular class (the majority class among observations in that subtree). The replacement is accepted if the resulting pruned tree does not perform worse than the original tree on a validation sample. This method is simple and effective but requires a separate validation dataset.

Q24: Cost-Complexity Pruning

What is cost-complexity pruning?

1. A method that adds a penalty term to RSS to trade off accuracy against number of terminal nodes.
2. A method that always increases the number of nodes.
3. A method that uses only entropy for pruning.
4. A method that ignores model complexity.

Answer: A Explanation: Cost-complexity pruning adds a penalty term to the RSS (or impurity measure) that increases with the number of terminal nodes. The objective becomes minimizing $RSS + \alpha \times$ (number of leaves) where α is a tuning parameter controlling the trade-off between training accuracy and model complexity. Cross-validation is typically used to select the optimal α value, balancing fit and simplicity.

Q25: Ensemble Techniques - Definition

What is the core idea behind ensemble techniques in machine learning?

1. Using a single complex model for prediction.
2. Combining many individual models to produce a single, superior meta-model.
3. Using only decision trees for prediction.
4. Removing all models and using manual prediction.

Answer: B Explanation: Ensemble techniques combine many individual models (often decision trees) to produce a single, superior meta-model. This approach leverages the "wisdom of crowds" principle: by aggregating predictions from diverse models, we can achieve better performance and reduce the influence of individual model errors. Ensembles typically improve overall accuracy and build in protection against overfitting. While they can combine any type of model, they are commonly illustrated with decision trees.

Q26: Bagging - Definition

What is bootstrap aggregation (bagging)?

1. A method that uses all data without sampling.
2. A method that creates multiple decision trees by bootstrapping samples from the training set, then aggregates predictions.
3. A method that uses only one decision tree.
4. A method that boosts the performance of a single tree.

Answer: B Explanation: Bootstrap aggregation (bagging) creates multiple decision trees by repeatedly sampling subsets from the training set with replacement (bootstrapping). Each tree is constructed on a different bootstrap sample. For regression problems, predictions are averaged across trees. For classification, a majority vote determines the final prediction. This averaging reduces variance and improves stability compared to a single tree.

Q27: Bagging - Sampling Method

What type of sampling is used in bagging?

1. Sampling without replacement.
2. Sampling with replacement (bootstrapping).
3. Random sampling without any replacement.
4. Stratified sampling.

Answer: B Explanation: Bagging uses sampling with replacement, also known as bootstrapping. This means that an observation can be selected multiple times in the same bootstrap sample, while other observations may not be selected at all. The observations not selected in a particular replication are called "out-of-bag" data and can be used to evaluate model performance without needing a separate validation set.

Q28: Out-of-Bag Data

What is out-of-bag (OOB) data in bagging?

1. Data that is used for training.
2. Observations not selected in a bootstrap sample that can be used for evaluation.
3. Data from a different distribution.
4. Data that is always included in training.

Answer: B Explanation: Out-of-bag (OOB) data refers to observations that are NOT selected in a particular bootstrap sample during bagging. Since sampling is done with replacement, approximately 63

Q29: Pasting - Definition

How does pasting differ from bagging?

1. Pasting uses sampling with replacement; bagging uses sampling without replacement.
2. Pasting uses sampling without replacement; bagging uses sampling with replacement.
3. Pasting uses all data; bagging uses subsets.
4. Pasting uses only one tree; bagging uses multiple trees.

Answer: B Explanation: Pasting is identical to bagging except that sampling is done without replacement. This means each data point can be drawn at most once in any replication. With a training set of 100,000 items and sub-samples of 10,000, there would be a total of 10 non-overlapping sub-samples. This reduces the diversity of the ensemble compared to bagging but may be computationally more efficient.

Q30: Bagging - Aggregation for Classification

How are predictions aggregated in bagging for classification problems?

1. By averaging the predicted values.
2. By taking a majority vote among the trees.
3. By selecting the prediction from the deepest tree.
4. By randomly selecting a prediction.

Answer: B Explanation: For classification problems in bagging, predictions are aggregated by majority vote. Each tree in the ensemble predicts a class for the new instance, and the class predicted by the majority of trees becomes the final prediction. This approach reduces the impact of individual tree errors and typically results in more robust and accurate predictions than using a single tree.

Q31: Random Forests - Definition

What distinguishes random forests from bagging?

1. Random forests use only one feature at each split.
2. Random forests use a random subset of features at each split, reducing correlation among trees.
3. Random forests use no bootstrapping.
4. Random forests use all features at each split.

Answer: B Explanation: Random forests are a refinement of bagging that adds an extra layer of randomness. While bagging uses all features for each split, random forests restrict each split to consider only a random subset of features (typically about $\sqrt{p}$ where p is the total number of features). This forces trees to be different from one another, reducing correlation among them. The resulting "forest" of diverse trees yields improved accuracy and reduced variance when predictions are aggregated.

Q32: Random Forest - Feature Subset Size

What is the typical number of features considered at each split in a random forest?

1. All features.
2. Approximately one feature.
3. Approximately $\sqrt{p}$ where p is the total number of features.
4. Approximately $p/2$ where p is the total number of features.

Answer: C Explanation: In a random forest, each split considers only a random subset of features, typically approximately $\sqrt{p}$ where p is the total number of features. This hyperparameter, often denoted as m_{try}, is crucial for decorrelating the trees. If p is large, using $\sqrt{p}$ features forces diversity among trees. If $m_{try} = p$, the random forest becomes equivalent to bagging, losing the benefit of reduced correlation.

Q33: Random Forest - Advantage

Why do random forests often outperform bagging?

1. Because they use more data.
2. Because they reduce correlation among trees, leading to better aggregated predictions.
3. Because they use deeper trees.
4. Because they use fewer features overall.

Answer: B Explanation: Random forests often outperform bagging because they reduce correlation among individual trees. By restricting each split to a random subset of features, the trees become less similar to each other. When uncorrelated trees are aggregated, the ensemble's variance is reduced more effectively than when correlated trees are averaged. This makes random forests one of the most effective "off-the-shelf" machine learning models.

Q34: Boosting - Definition

What is boosting in ensemble learning?

1. Growing multiple trees independently and in parallel.
2. Growing trees sequentially, where each new tree corrects the errors of the previous tree.
3. Using only one tree with increased depth.
4. Removing trees from the ensemble.

Answer: B Explanation: Boosting is a sequential ensemble technique where each new tree is grown to correct the errors of the previously grown trees. Unlike bagging which grows trees independently and in parallel, boosting builds models iteratively. Each subsequent model focuses on the observations that were misclassified by previous models, leading to a progressively improving ensemble. This focused learning process makes boosting extremely powerful but also more prone to overfitting.

Q35: Gradient Boosting - How It Works

How does gradient boosting work?

1. Each new tree is fit to the original target variable.
2. Each new tree is fit to the residuals (errors) of the combined previous model.
3. Each new tree is fit to a random subset of features.
4. Each new tree is fit without any reference to previous trees.

Answer: B Explanation: Gradient boosting fits each new tree to the residuals (errors) of the combined model from previous iterations. First, a simple tree is fit to the data. Then, the residuals (actual minus predicted) are calculated, and a second tree is fit to these residuals. The predictions are then combined. This process repeats, with each subsequent tree focusing on the errors of the combined model. This iterative approach slowly improves the model by targeting areas of poor performance.

Q36: AdaBoost - How It Works

How does AdaBoost (Adaptive Boosting) work?

1. It gives equal weights to all observations throughout the process.
2. It increases the weights of misclassified observations so subsequent trees focus on them.
3. It decreases the weights of misclassified observations.
4. It uses no weights in the training process.

Answer: B Explanation: AdaBoost starts with equal weights for all observations. After training a simple tree, it examines the errors. Observations that were misclassified have their weights increased. A second tree is then trained, incentivized to pay more attention to these higher-weight observations. This process of re-weighting and re-training repeats sequentially. The final model is a weighted sum of all trees, where better-performing trees receive more weight.

Q37: Boosting - Risk

What is a major risk of boosting compared to random forests?

1. It is less accurate on all data.
2. It is more prone to overfitting if not carefully tuned.
3. It requires more features.
4. It uses more memory.

Answer: B Explanation: Boosting models are extremely powerful and often achieve state-of-the-art performance, but they are more prone to overfitting than random forests. Because boosting builds models sequentially and focuses on errors, it can become overly specialized to the training data if not carefully controlled. Mitigation strategies include limiting the number of trees, restricting tree depth, and using learning rate (shrinkage) to slow down the learning process.

Q38: Boosting vs Bagging - Key Difference

What is the key difference between boosting and bagging?

1. Bagging uses decision trees; boosting uses other models.
2. Bagging grows trees in parallel; boosting grows trees sequentially.
3. Bagging uses sampling; boosting uses no sampling.
4. Bagging is for regression; boosting is for classification.

Answer: B Explanation: The key difference between boosting and bagging is that bagging grows multiple trees independently and in parallel, while boosting grows trees sequentially. In boosting, each new tree is built to correct the errors of the previous trees. Bagging reduces variance by averaging many independent models; boosting reduces bias by sequentially focusing on difficult observations. This sequential nature makes boosting more powerful but also more prone to overfitting.

Q39: Ensemble Technique - Which Returns Best Results?

Which statement about ensemble techniques is most accurate?

1. Ensembles never outperform single models.
2. Ensembles usually return the best model results among available techniques.
3. Ensembles always overfit the data.
4. Ensembles should only be used with decision trees.

Answer: B Explanation: While not a hard rule, ensemble techniques usually return the best model results among available techniques. By aggregating predictions from diverse models, ensembles typically achieve superior accuracy and robustness compared to single models. This is why random forests, gradient boosting, and other ensemble methods are often preferred in practice, despite their reduced interpretability compared to single decision trees.

Q40: Ensemble - Advantages Summary

What are the two primary objectives of ensemble techniques?

1. To reduce model size and increase speed.
2. To improve overall model accuracy and build in protection against overfitting.
3. To decrease interpretability and increase complexity.
4. To use less data and reduce computation.

Answer: B Explanation: The two primary objectives of ensemble techniques are: (1) to improve overall model fit and accuracy by averaging out individual model errors, and (2) to build in protection against overfitting by combining diverse models. While ensembles may reduce interpretability, their superior predictive performance often makes them the preferred choice in practice, especially for complex problems where accuracy is paramount.

Q41: KNN - Definition

What is K-Nearest Neighbors (KNN)?

1. A supervised learning model that builds a complex mathematical function during training.
2. A supervised learning model that memorizes the training data and makes predictions based on the K closest instances.
3. An unsupervised learning model used for clustering.
4. A reinforcement learning model for sequential decisions.

Answer: B Explanation: K-Nearest Neighbors (KNN) is a supervised learning model that memorizes the entire training dataset.

When a prediction is requested for a new instance, it identifies the K training instances closest to it using a distance metric (like Euclidean distance). For classification, a majority vote among these neighbors determines the class; for regression, the average of their values is returned. It is sometimes called a "lazy learner" because no explicit training phase occurs.

Q42: KNN - "Lazy Learner"

Why is KNN sometimes called a "lazy learner"?

1. Because it trains slowly.
2. Because it doesn't build an explicit, general model during training; all "work" happens at prediction time.
3. Because it uses lazy evaluation in programming.
4. Because it always chooses the easiest classification.

Answer: B Explanation: KNN is called a "lazy learner" (or instance-based learner) because it does not construct an explicit, general model during a training phase. Unlike regression or decision trees that learn a function during training, KNN simply stores the entire training dataset. The actual computation happens only when a prediction for a new instance is requested, at which point it compares the new instance to all training instances to find the nearest neighbors. This means there is no training cost but high prediction cost.

Q43: KNN - Distance Metric

Which distance metric is most commonly used in KNN?

1. Manhattan distance.
2. Euclidean distance.
3. Cosine similarity.
4. Hamming distance.

Answer: B Explanation: The Euclidean distance is the most commonly used distance metric in KNN. It measures the straight-line distance between two points in the feature space. The formula is $d_E = \sqrt{\sum_{j=1}^{m}(x_{1j} - x_{2j})^2}$ where m is the number of features. Other metrics like Manhattan distance and cosine similarity can also be used, but Euclidean distance is the default choice in most implementations due to its intuitive geometric interpretation.

Q44: KNN - Feature Scaling Requirement

Why is feature scaling a crucial prerequisite for KNN?

1. Because KNN requires all features to be integers.
2. Because features with larger scales dominate the distance calculation, potentially biasing the results.
3. Because KNN cannot handle categorical features.
4. Because KNN requires features to be normally distributed.

Answer: B Explanation: Feature scaling is crucial for KNN because the algorithm relies entirely on distance calculations. If one feature (e.g., income in thousands of dollars) has a much larger scale than another (e.g., age in years), the larger-scale feature will dominate the distance metric, effectively ignoring the smaller-scale feature. Standardization (mean 0, standard deviation 1) or normalization (range 0 to 1) ensures all features contribute equally to the distance calculation.

Q45: KNN - Choosing K

What is a common heuristic for choosing K in KNN?

1. $K = N$ where N is the total number of training samples.
2. $K = \sqrt{N}$ where N is the total number of training samples.
3. $K = \log(N)$ where N is the total number of training samples.
4. $K = 1$ always.

Answer: B Explanation: A common heuristic is to set $K \approx \sqrt{N}$ where N is the total size of the training sample. For example, if $N = 5,000$, set $K = \sqrt{5000} \approx 71$. Small values of K tend to overfit (capturing noise), while large values of K may underfit (smoothing over important patterns). Cross-validation can also be used to tune K and choose the value that minimizes error on a validation sample.

Q46: KNN - Small K Effects

What is the effect of using a small value of K in KNN?

1. It tends to underfit the data.
2. It tends to overfit the data.
3. It has no effect on the model.
4. It always improves accuracy.

Answer: B Explanation: Small values of K (e.g., K=1) tend to overfit the data because the model becomes sensitive to local noise and outliers. With K=1, a single noisy training point can significantly influence predictions. As K increases, the decision boundary becomes smoother and more robust, reducing variance but potentially increasing bias (underfitting). Cross-validation helps find the optimal K that balances this bias-variance trade-off.

Q47: KNN - Large K Effects

What is the effect of using a large value of K in KNN?

1. It tends to overfit the data.
2. It tends to underfit the data.
3. It has no effect on the model.
4. It always improves accuracy.

Answer: B Explanation: Large values of K tend to underfit the data because the model averages over too many points, smoothing out important patterns in the data. If K is too large (e.g., close to the total number of training samples), the model essentially predicts the global average for regression or the global majority class for classification, ignoring local structure. The optimal K balances the bias-variance trade-off through validation.

Q48: KNN - Disadvantages

What is a significant disadvantage of KNN?

1. It builds an explicit model during training.
2. It is computationally intensive because it must compute distances between the new instance and all training instances.
3. It cannot handle continuous features.
4. It always overfits the data.

Answer: B Explanation: KNN's major disadvantage is its computational intensity. For each prediction, distances between the new instance and all training instances must be computed, making it $O(N \times m)$ where N is the number of training samples and m is the number of features. This makes KNN impractical for large datasets. Various optimization techniques (like KD-trees or Ball-trees) can speed up the search, but the basic algorithm remains computationally expensive.

Q49: KNN - Another Disadvantage

When does KNN perform poorly?

1. When there are many features and some are irrelevant or noisy.
2. When the data is linearly separable.
3. When there are few training examples.
4. When the data is perfectly clean.

Answer: A Explanation: KNN performs poorly when there are irrelevant or noisy features because these features can drive similar instances apart in the feature space, distorting the distance metric. With many irrelevant features, the "curse of dimensionality" makes distance measures less meaningful. Feature selection or dimensionality reduction (like PCA) can help mitigate this issue by focusing on the most informative features.

Q50: KNN - Worked Example

In the KNN borrower example with K=4, the four nearest neighbors were observations 8, 10, 4, and 9. Three of these were defaulted borrowers. How was the new instance classified?

1. As Non-Default (0) because one neighbor was Non-Default.
2. As Default (1) because of majority vote.
3. As Default (1) because of average of labels.
4. As Non-Default (0) because average label was below 0.5.

Answer: B Explanation: With K=4, the four nearest neighbors were observations 8, 10, 4, and 9. Among these, three were defaulted borrowers (class 1) and one was non-defaulted (class 0). Using majority voting, the new instance is classified as Default (1) because the majority of its nearest neighbors belong to that class. This demonstrates the fundamental classification mechanism of KNN.

Q51: SVM - Definition

What is the core objective of a Support Vector Machine (SVM)?

1. To minimize the number of features used.
2. To find the "best" possible boundary (hyperplane) that separates data points belonging to different classes.
3. To maximize the number of support vectors.
4. To minimize the training time.

Answer: B Explanation: The core objective of SVM is to find the optimal separating hyperplane that maximizes the margin between different classes. The "best" boundary is the one that creates the widest possible margin between classes, providing a robust and confident separation. This maximum margin classifier generalizes better to new data because it places the decision boundary as far as possible from the training points.

Q52: SVM - Maximum Margin Classifier

What is the maximum margin classifier in SVM?

1. The line that perfectly separates all training points with the smallest margin.
2. The hyperplane that maximizes the distance (margin) between the boundary and the closest points from each class.
3. The line that passes through the support vectors.
4. The hyperplane that minimizes classification error.

Answer: B Explanation: The maximum margin classifier is the hyperplane that maximizes the distance (margin) between the decision boundary and the closest points from each class. The boundary runs down the middle of this margin. A wider margin provides more robust separation and better generalization. The data points that lie exactly on the edges of the margin are called support vectors, as they "support" the definition of the optimal boundary.

Q53: SVM - Support Vectors

What are support vectors in SVM?

1. All data points in the training set.
2. The data points from the training set that are closest to the classification boundary, lying on the margin edges.
3. The data points that are farthest from the classification boundary.
4. The data points that were misclassified.

Answer: B Explanation: Support vectors are the critical data points from the training set that lie exactly on the edges of the margin (the closest points to the decision boundary). These are the most important points because they "support" or define the position and orientation of the margin and decision boundary. If any support vector were to move, the optimal boundary would also move. Points far from the boundary have no influence on the final boundary.

Q54: SVM - Support Vectors vs Other Points

Why do only support vectors influence the SVM decision boundary?

1. Because only support vectors are used for training.
2. Because points far from the boundary don't affect the margin's position; only the closest points matter.
3. Because all points are equally weighted.
4. Because support vectors are the only points with correct labels.

Answer: B Explanation: In SVM, only the support vectors influence the decision boundary. Points that are far away from the boundary and well within their class region do not affect the margin's position. The optimization problem focuses on maximizing the margin, which is determined solely by the closest points to the boundary—the support vectors. This makes SVM robust and efficient, as the solution depends on a subset of the training data.

Q55: SVM - Hyperplane Equation

In a 2D SVM, what does the equation $w_0 + w_1 x_1 + w_2 x_2 = 0$ represent?

1. The margin constraint.
2. The maximum margin classifier (decision boundary).
3. The support vector location.
4. The training data distribution.

Answer: B Explanation: In a 2D SVM, the equation $w_0 + w_1 x_1 + w_2 x_2 = 0$ represents the maximum margin classifier (decision boundary). This is the line that runs exactly in the middle of the margin. The parameters w_0, w_1, and w_2 are learned during training to maximize the margin while correctly separating the classes. The margin constraints are $w_0 + w_1 x_1 + w_2 x_2 = \pm 1$ for classes $+1$ and -1.

Q56: SVM - Margin Width Formula

What is the formula for the margin width in a 2D SVM?

1. $2/|w|$ where $|w| = \sqrt{w_1^2 + w_2^2}$
2. $1/|w|$ where $|w| = \sqrt{w_1^2 + w_2^2}$
3. $|w|/2$ where $|w| = \sqrt{w_1^2 + w_2^2}$
4. $|w|$ where $|w| = \sqrt{w_1^2 + w_2^2}$

Answer: A Explanation: The margin width in a 2D SVM is $2/|w|$ where $|w| = \sqrt{w_1^2 + w_2^2}$ is the Euclidean norm of the weight vector. Maximizing the margin width is equivalent to minimizing $|w|$. This is why the SVM optimization problem is often expressed as minimizing $\frac{1}{2}|w|^2$ subject to the constraint that all points lie outside the margin. A smaller $|w|$ corresponds to a wider margin and better generalization.

Q57: SVM - Optimization Objective

What is the SVM optimization problem for linearly separable data?

1. Minimize $\frac{1}{2}|w|^2$ subject to $y_j(w_0 + w_1x_{1j} + w_2x_{2j}) \geq 1$ for all j.
2. Maximize $\frac{1}{2}|w|^2$ subject to $y_j(w_0 + w_1x_{1j} + w_2x_{2j}) \leq 1$ for all j.
3. Minimize $|w|$ subject to $y_j(w_0 + w_1x_{1j} + w_2x_{2j}) \geq 0$ for all j.
4. Maximize $|w|$ subject to $y_j(w_0 + w_1x_{1j} + w_2x_{2j}) = 1$ for all j.

Answer: A Explanation: The SVM optimization problem for linearly separable data is to minimize $\frac{1}{2}|w|^2$ subject to $y_j(w_0 + w_1x_{1j} + w_2x_{2j}) \geq 1$ for all j, where $y_j = +1$ for class 1 and $y_j = -1$ for class 2. Minimizing $\frac{1}{2}|w|^2$ is equivalent to maximizing the margin $2/|w|$. The constraints ensure all points lie on or outside the margin boundaries.

Q58: SVM - Soft Margin

What is a soft margin in SVM?

1. A margin that is always perfectly separating the classes.
2. A margin that allows some points to be misclassified or fall within the margin, controlled by a tuning parameter λ.
3. A margin that uses only linear boundaries.
4. A margin that ignores support vectors.

Answer: B Explanation: A soft margin allows the SVM to be more flexible by permitting some points to be misclassified or to fall within the margin. A tuning parameter λ (or C) controls the trade-off between a wide margin and the number of misclassifications. A large λ allows a wider margin even with more misclassifications (more "tolerant"), while a small λ penalizes misclassification heavily (less tolerant). This makes SVM applicable to data that is not perfectly separable.

Q59: SVM - Soft Margin Loss Function

What is the loss function for SVM with a soft margin?

1. $L(w) = \frac{1}{N} \sum_{j=1}^{N} \max(0, 1 - y_j(w_0 + w^T x_j)) + \frac{\lambda}{2} |w|^2$
2. $L(w) = \frac{1}{N} \sum_{j=1}^{N} \max(0, 1 - y_j(w_0 + w^T x_j))$
3. $L(w) = \frac{1}{N} \sum_{j=1}^{N} (y_j - (w_0 + w^T x_j))^2 + \frac{\lambda}{2} |w|^2$
4. $L(w) = \frac{1}{N} \sum_{j=1}^{N} |y_j - (w_0 + w^T x_j)| + \frac{\lambda}{2} |w|^2$

Answer: A Explanation: The soft margin SVM loss function is $L(w) = \frac{1}{N} \sum_{j=1}^{N} \max(0, 1 - y_j(w_0 + w^T x_j)) + \frac{\lambda}{2} |w|^2$. The first term is the hinge loss, which is 0 for correctly classified points outside the margin and equals the distance to the margin for misclassified points. The second term is the regularization term (like ridge regression) that controls the margin width. The hyperparameter λ controls the trade-off between margin width and misclassification penalty.

Q60: SVM - Hinge Loss

What is the hinge loss in SVM?

1. A loss function that penalizes all points equally.
2. A loss function that is zero for correct classifications outside the margin and increases linearly for points on the wrong side of the margin.
3. A loss function that is always positive.
4. A loss function that uses squared errors.

Answer: B Explanation: The hinge loss is defined as $\max(0, 1 - y_j(w_0 + w^T x_j))$. It is zero for points that are correctly classified and outside the margin (i.e., $y_j(w_0 + w^T x_j) \geq 1$). For points that are misclassified or fall within the margin, the loss increases linearly with the distance from the margin. This makes SVM robust to outliers and allows for soft margins when data is not perfectly separable.

Q61: SVM - Kernel Trick

What is the kernel trick in SVM?

1. A method to reduce the number of support vectors.
2. A mathematical technique that projects data into a higher-dimensional space to find a linear separator, without explicitly computing the projection.
3. A method to increase the margin width.
4. A technique to remove outliers from the dataset.

Answer: B Explanation: The kernel trick allows SVMs to create non-linear decision boundaries by implicitly projecting data into a higher-dimensional space where a linear separator exists. For example, data forming concentric circles in 2D might become linearly separable in 3D. The "trick" is that SVMs can perform these calculations and find the linear boundary in the higher-dimensional space using kernel functions (like polynomial or RBF kernels) without ever explicitly computing the coordinates in that space, which would be computationally expensive.

Q62: SVM - Kernel Example

Which of the following is a commonly used kernel in SVM?

1. Linear kernel only.
2. Polynomial kernel.
3. Radial Basis Function (RBF) kernel.
4. Both polynomial and RBF kernels.

Answer: D Explanation: Common kernels used in SVM include the linear kernel, polynomial kernel, and Radial Basis Function (RBF) kernel. The RBF kernel is the most popular for non-linear problems because it can create complex decision boundaries and has good theoretical properties. The choice of kernel depends on the data characteristics; linear kernels work for linearly separable data, while polynomial and RBF kernels handle non-linear relationships effectively.

Q63: SVM - Car Loan Example

In the car loan SVM example, with $w_0 = -12.24$, $w_1 = 0.90$, and $w_2 = 1.26$, a new borrower with income=5.7 and savings=3.5 had $D(x) = -2.7$. What was the classification?

1. Loan granted (+1) because D(x) ¿ 0.
2. Loan not granted (-1) because D(x) ¡ 0.
3. Cannot determine without more information.
4. Loan granted with high confidence.

Answer: B Explanation: In the car loan SVM example, loans granted were coded as +1 and loans not granted as -1. The decision boundary is given by $D(x) = w_0 + w_1(\text{income}) + w_2(\text{savings}) = 0$. For the new borrower, $D(x) = -12.24 + 0.90(5.7) + 1.26(3.5) = -2.7$, which is negative. Therefore, the loan was classified as not granted (-1). This demonstrates how the SVM's decision function determines class membership based on the sign of $D(x)$.

Q64: SVM - When to Use

SVMs are particularly well-suited for which type of problems?

1. Problems with very few features.
2. Classification problems, especially those with a large number of features.
3. Regression problems with continuous targets.
4. Unsupervised clustering problems.

Answer: B Explanation: SVMs are particularly well-suited for classification problems, especially those with a large number of features. They can handle high-dimensional data efficiently because the decision boundary depends only on the support vectors. SVMs are robust to overfitting in high-dimensional spaces and can capture complex non-linear relationships through the kernel trick, making them powerful tools for text classification, image recognition, and other complex classification tasks.

Q65: SVM - Key Advantage

What is a key advantage of SVMs over other classification methods?

1. They are always faster to train.
2. They are robust to overfitting, especially in high-dimensional spaces, due to the margin maximization principle.
3. They never require feature scaling.
4. They always produce interpretable models.

Answer: B Explanation: A key advantage of SVMs is their robustness to overfitting, particularly in high-dimensional spaces. The margin maximization principle provides a natural form of regularization—the wider the margin, the better the generalization. Additionally, the use of support vectors means the solution depends only on a subset of training points, making SVMs efficient and robust. However, SVMs can be computationally intensive for very large datasets and may not be interpretable when using non-linear kernels.

Q66: Neural Network - Definition

What are Artificial Neural Networks (ANNs)?

1. A class of models based on linear regression.
2. A class of machine learning models loosely inspired by biological neural networks in the human brain.
3. A class of unsupervised learning algorithms for clustering.
4. A class of models that only work for regression problems.

Answer: B Explanation: Artificial Neural Networks (ANNs) are machine learning models loosely inspired by the biological neural networks in human brains. They consist of interconnected nodes (neurons) organized in layers. The most common type is the feedforward network or Multi-Layer Perceptron (MLP), where information flows from input layer through hidden layers to output layer. Neural networks are versatile and can be used for both classification and regression tasks.

Q67: MLP - Layers

What are the three types of layers in a Multi-Layer Perceptron (MLP)?

1. Input layer, hidden layer(s), output layer.
2. Input layer, regression layer, output layer.
3. Hidden layer, activation layer, output layer.
4. Input layer, activation layer, loss layer.

Answer: A Explanation: A Multi-Layer Perceptron (MLP) consists of: (1) an input layer that receives raw data (one neuron per feature), (2) one or more hidden layers where most computation occurs, and (3) an output layer that produces the final prediction. The hidden layers are where the network learns complex, non-linear relationships. Information flows in a "feedforward" manner from input to output.

Q68: Neural Network - Weights

What are the weights in a neural network?

1. The fixed values that do not change during training.
2. The parameters that multiply the inputs and are learned during training.
3. The activation functions applied to each neuron.
4. The number of neurons in each layer.

Answer: B Explanation: Weights are the parameters that multiply the inputs (or outputs from the previous layer) in a neural network. They are initially random and are adjusted during training through backpropagation to minimize the loss function. Each connection between neurons has an associated weight that amplifies or dampens the signal passing through it. Learning in a neural network is primarily about finding the optimal values of these weights.

Q69: Hidden Layer Calculation

How is the value of a hidden neuron calculated?

1. By taking the maximum of the input values.
2. By applying an activation function to the weighted sum of inputs plus a bias term.
3. By averaging the input values.
4. By random selection.

Answer: B Explanation: A hidden neuron's value is calculated by: (1) taking the weighted sum of inputs from the previous layer plus a bias term, and (2) applying a non-linear activation function to this sum. Mathematically, $H_j = f(\sum_i w_{ij} x_i + b_j)$ where f is the activation function, w_{ij} are weights, x_i are inputs, and b_j is the bias. This non-linear transformation is what enables neural networks to learn complex patterns.

Q70: Activation Function - Purpose

Why are non-linear activation functions essential in neural networks?

1. To make the network faster.
2. To allow the network to learn non-linear relationships; without them, the entire network would be equivalent to a linear model.
3. To reduce the number of weights.
4. To improve interpretability.

Answer: B Explanation: Non-linear activation functions are essential because without them, a neural network with any number of layers would simply be performing a series of linear transformations, making the entire network equivalent to a single linear regression model. The non-linearity introduced by activation functions (like ReLU, sigmoid, tanh) allows neural networks to learn complex, non-linear relationships in the data, which is the key to their power and versatility.

Q71: Sigmoid Activation Function

What is the range of the sigmoid (logistic) activation function?

1. $(-\infty, \infty)$
2. $(0, 1)$
3. $(-1, 1)$
4. $[0, \infty)$

Answer: B Explanation: The sigmoid (logistic) activation function outputs values between 0 and 1. The formula is $f(z) = \frac{1}{1+e^{-z}}$. This range makes it particularly useful for binary classification problems where the output can be interpreted as a probability. However, sigmoid can suffer from vanishing gradients for large positive or negative inputs, which can slow down learning in deep networks.

Q72: Sigmoid Formula

What is the formula for the sigmoid (logistic) activation function?

1. $f(z) = \frac{1}{1+e^{-z}}$
2. $f(z) = \frac{1}{1+e^{z}}$
3. $f(z) = \frac{e^{z}}{1+e^{z}}$
4. $f(z) = \tanh(z)$

Answer: A Explanation: The sigmoid activation function is $f(z) = \frac{1}{1+e^{-z}}$, where e is the base of the natural logarithm. This is the same function used in logistic regression. It maps any real-valued input to a value between 0 and 1. The sigmoid function is S-shaped (monotonic) and differentiable, making it suitable for gradient-based optimization methods like backpropagation.

Q73: ReLU Activation Function

What is the ReLU activation function?

1. $f(z) = \max(z, 0)$
2. $f(z) = \min(z, 0)$
3. $f(z) = \frac{1}{1+e^{-z}}$
4. $f(z) = \tanh(z)$

Answer: A Explanation: The ReLU (Rectified Linear Unit) activation function is $f(z) = \max(z, 0)$. It returns 0 for negative inputs and the input value itself for positive inputs. ReLU is computationally efficient and helps mitigate the vanishing gradient problem that affects sigmoid and tanh functions. It is the most commonly used activation function for hidden layers in modern neural networks, including deep learning architectures.

Q74: Leaky ReLU

What is the Leaky ReLU activation function?

1. $f(z) = \max(z, 0)$
2. $f(z) = \begin{cases} 0.01z & z < 0 \\ z & z \geq 0 \end{cases}$
3. $f(z) = \begin{cases} 0 & z < 0 \\ z & z \geq 0 \end{cases}$
4. $f(z) = \frac{1}{1+e^{-z}}$

Answer: B Explanation: Leaky ReLU is a variation of ReLU that allows small negative values when the input is negative: $f(z) = \begin{cases} 0.01z & z < 0 \\ z & z \geq 0 \end{cases}$. Unlike standard ReLU which outputs 0 for all negative inputs, Leaky ReLU allows a small, non-zero gradient for negative inputs. This prevents "dying ReLU" where neurons become permanently inactive, potentially improving learning in deep networks.

Q75: Tanh Activation Function

What is the range of the hyperbolic tangent (tanh) activation function?

1. $(0, 1)$
2. $(-1, 1)$
3. $(-\infty, \infty)$
4. $[0, \infty)$

Answer: B Explanation: The hyperbolic tangent (tanh) activation function outputs values between -1 and 1. The formula is $f(z) = \frac{e^{2z}-1}{e^{2z}+1}$. Tanh is zero-centered, which can help with optimization by centering the data. It has a similar S-shape to the sigmoid function but outputs negative values, making it useful for networks where negative activations are meaningful. However, it can also suffer from vanishing gradients for large inputs.

Q76: Softmax Activation Function

What is the softmax activation function used for?

1. Binary classification problems.
2. Multi-class classification problems, converting raw scores to probabilities that sum to 1.
3. Regression problems.
4. Dimensionality reduction.

Answer: B Explanation: The softmax function is used for multi-class classification problems. It takes a vector of raw scores (logits) and converts them to probabilities that sum to 1. The formula is $f(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$ where K is the number of classes. Softmax exponentially magnifies the largest input values, making it easy to interpret the outputs as class probabilities. It is typically used at the output layer for multi-class problems.

Q77: Backpropagation

What is backpropagation?

1. The process of initializing weights randomly.
2. The algorithm that propagates errors backward through the network to update weights and minimize the loss function.
3. The process of feeding data forward through the network.
4. The method of selecting the number of hidden layers.

Answer: B Explanation: Backpropagation is the core training algorithm for neural networks. It works by: (1) performing a forward pass to make a prediction, (2) calculating the error (loss) between the prediction and actual value, and (3) propagating this error backward through the network, using the chain rule to calculate gradients of the loss with respect to each weight. These gradients are then used to update weights in the direction that minimizes the loss. This process repeats for many iterations (epochs) until convergence.

Q78: Backpropagation - Forward Pass

What happens during the forward pass in neural network training?

1. The network calculates errors and updates weights.
2. The input data passes through the network to generate a prediction.
3. The network selects the optimal number of layers.
4. The network applies pruning to reduce complexity.

Answer: B Explanation: During the forward pass, input data flows through the network from the input layer, through the hidden layers, to the output layer. Each neuron applies its activation function to the weighted sum of its inputs. The output layer produces the network's prediction. This prediction is then compared to the actual target value to calculate the loss (error). The forward pass is the first step in the backpropagation process, followed by the backward pass where errors are propagated and weights updated.

Q79: Epochs in Neural Networks

What is an epoch in neural network training?

1. A single forward pass through the network.
2. One complete pass of the entire training dataset through the network (forward and backward).
3. The number of hidden layers in the network.
4. The learning rate used for optimization.

Answer: B Explanation: An epoch is one complete pass of the entire training dataset through the neural network, including both the forward pass (to make predictions) and the backward pass (to update weights). Typically, neural networks are trained for multiple epochs (e.g., 100, 1000) until the loss converges or stops improving. The number of epochs is an important hyperparameter; too few can lead to underfitting, while too many can lead to overfitting.

Q80: Universal Approximation Theorem

What does the Universal Approximation Theorem state?

1. Neural networks can only approximate linear functions.
2. With enough hidden neurons, a neural network can approximate any continuous function with arbitrary precision.
3. Neural networks always overfit the data.
4. Neural networks are only suitable for classification problems.

Answer: B Explanation: The Universal Approximation Theorem states that a feedforward neural network with a single hidden layer containing a finite number of neurons can approximate any continuous function on a compact set with arbitrary precision, assuming appropriate activation functions. This theoretical result explains why neural networks are so powerful and flexible. However, the theorem does not guarantee that the network can be trained effectively—that depends on practical considerations like data availability, training algorithms, and avoiding overfitting.

Q81: Overfitting in Neural Networks

Why are neural networks particularly prone to overfitting?

1. Because they have too few parameters.
2. Because of their great flexibility and many parameters, they can learn random artifacts in the training data.
3. Because they always use linear activation functions.
4. Because they are slow to train.

Answer: B Explanation: Neural networks are particularly prone to overfitting because their great flexibility (many parameters) allows them to memorize the training data, including noise and random artifacts. This is the "double-edged sword" of neural networks—their power is also their vulnerability. Overfitting is especially problematic when the model is large and training data is limited. Signs of overfitting include high training accuracy but poor test accuracy, and different predictions from different training samples.

Q82: Dropout - Definition

What is dropout in neural networks?

1. Removing neurons from the network permanently.
2. An ensemble technique that selectively drops a few neurons during training, creating alternative networks whose predictions are aggregated.
3. A method to increase the number of neurons.
4. A technique to speed up training by reducing the number of epochs.

Answer: B Explanation: Dropout is an ensemble technique designed to reduce overfitting in neural networks. During training, a random subset of neurons is "dropped out" (set to zero) at each iteration, effectively creating multiple different network architectures. The predictions from these alternative networks are aggregated to produce the final prediction. This prevents the network from becoming overly dependent on any single neuron and encourages more robust feature learning.

Q83: Early Stopping

What is early stopping in neural network training?

1. Stopping training when the training loss reaches zero.
2. Stopping training before convergence on the training data, using a hold-out validation set to determine the optimal stopping point.
3. Stopping training after a fixed number of epochs regardless of performance.
4. Stopping training when all weights become zero.

Answer: B Explanation: Early stopping involves monitoring performance on a hold-out validation set during training. Training is stopped when the validation error starts to increase, even though the training error continues to decrease. This prevents overfitting by finding the point where the model generalizes best. Early stopping is a form of regularization that is simple to implement and effective in practice, commonly used with neural networks and other iterative algorithms.

Q84: Overfitting Prevention Methods

Which of the following is NOT a method to prevent overfitting in neural networks?

1. Penalty-based regularization.
2. Dropout.
3. Increasing the number of hidden layers.
4. Early stopping.

Answer: C Explanation: Increasing the number of hidden layers generally increases model complexity, which can make overfitting worse, not better. Methods to prevent overfitting include: (1) penalty-based regularization (adding a penalty term to the loss function), (2) dropout (randomly dropping neurons during training), (3) early stopping (monitoring validation error), and (4) using more training data or data augmentation. Adding more layers without regularization typically exacerbates overfitting.

Q85: CNN - Definition

What is a Convolutional Neural Network (CNN) designed for?

1. Time series data only.
2. Data with grid-like topology, such as images.
3. Text data only.
4. Structured tabular data.

Answer: B Explanation: Convolutional Neural Networks (CNNs) are designed to work with data that has a grid-like topology, such as images (2D grid of pixels). They use specialized convolutional layers with filters (kernels) that slide across the input, creating feature maps. While images are their most famous application, CNNs can also be applied to text (1D grid of words) and some time-series data. Their key innovations are translation invariance and parameter sharing, making them efficient for high-dimensional grid data.

Q86: Autoencoder - Definition

What is an autoencoder?

1. A supervised learning model for classification.
2. A neural network used for unsupervised learning, primarily for dimensionality reduction.
3. A clustering algorithm like K-means.
4. A regression model for continuous prediction.

Answer: B Explanation: An autoencoder is a type of artificial neural network used for unsupervised learning, primarily for dimensionality reduction. It consists of an encoder that compresses the input into a lower-dimensional representation (code) and a decoder that reconstructs the original input from this compressed representation. The network is trained to minimize reconstruction error, forcing it to learn the most important features of the data.

Q87: Autoencoder - Architecture

What are the two main parts of an autoencoder?

1. Input layer and output layer only.
2. Encoder (compresses input to hidden layer) and decoder (reconstructs input from hidden layer).
3. Hidden layer and classification layer.
4. Feature extraction layer and prediction layer.

Answer: B Explanation: An autoencoder has two main parts: (1) an encoder that maps the high-dimensional input to a lower-dimensional hidden layer (the "code" or latent representation), and (2) a decoder that maps this compressed representation back to the original feature space. The goal is to make the reconstructed output as close as possible to the original input. The hidden layer has fewer neurons than the input layer, forcing compression and efficient representation learning.

Q88: Autoencoder - Bottleneck

What is the "bottleneck" in an autoencoder?

1. The output layer with multiple neurons.
2. The hidden layer(s) with fewer neurons than the input layer, forcing compression.
3. The activation function used in the network.
4. The loss function used for training.

Answer: B Explanation: The "bottleneck" refers to the hidden layer(s) in an autoencoder that have fewer neurons than the input layer. This bottleneck forces the network to learn a compressed, efficient representation of the data. The network must find the most important features to reconstruct the input accurately, discarding noise and redundancy. The size of the bottleneck (number of hidden neurons) controls the degree of dimensionality reduction.

Q89: Autoencoder - Reconstruction Error

How is an autoencoder evaluated?

1. By classification accuracy.
2. By reconstruction error (the difference between original input and reconstructed output).
3. By the number of epochs trained.
4. By the number of hidden layers.

Answer: B Explanation: Autoencoders are evaluated by their reconstruction error—the difference between the original input and the reconstructed output. Common loss functions include Mean Squared Error (MSE) and Residual Sum of Squares (RSS). A low reconstruction error indicates that the autoencoder has learned a good compressed representation that captures the essential features of the data. High reconstruction error suggests important information was lost during compression.

Q90: Autoencoder - MSE Formula

What is the Mean Squared Error (MSE) loss function for an autoencoder?

1. $L = \sum_{j=1}^{m} \sum_{i=1}^{N} (x_{ij} - \hat{x}_{ij})$
2. $L = \frac{1}{mN} \sum_{j=1}^{m} \sum_{i=1}^{N} (x_{ij} - \hat{x}_{ij})^2$
3. $L = \sum_{j=1}^{m} \sum_{i=1}^{N} |x_{ij} - \hat{x}_{ij}|$
4. $L = \frac{1}{mN} \sum_{j=1}^{m} \sum_{i=1}^{N} |x_{ij} - \hat{x}_{ij}|$

Answer: B Explanation: The Mean Squared Error (MSE) loss for an autoencoder is $L = \frac{1}{mN} \sum_{j=1}^{m} \sum_{i=1}^{N} (x_{ij} - \hat{x}_{ij})^2$, where m is the number of features, N is the number of observations, x_{ij} are the original features, and $\hat{x}_{ij}$ are the reconstructed features. MSE averages the squared differences between original and reconstructed values. A smaller MSE indicates better reconstruction and a more effective compressed representation.

Q91: Autoencoder vs PCA

How do autoencoders differ from PCA?

1. Autoencoders are linear; PCA is non-linear.
2. Autoencoders use neural networks and can learn non-linear relationships; PCA is linear.
3. Autoencoders and PCA are identical.
4. PCA requires labeled data; autoencoders don't.

Answer: B Explanation: Autoencoders differ from PCA in that they can learn non-linear relationships through non-linear activation functions. PCA is limited to linear transformations, finding orthogonal components that maximize variance. Autoencoders, on the other hand, can capture complex, non-linear patterns in the data. With non-linear activation functions, autoencoders can often achieve better compression than PCA, especially for data with intricate structures. PCA has a unique mathematical solution; autoencoders are trained through optimization.

Q92: Autoencoder - Anomaly Detection

How are autoencoders used for anomaly detection?

1. By training on labeled anomalies.
2. By training on "normal" data and flagging instances with high reconstruction error as anomalies.
3. By classifying normal and anomalous instances.
4. By using the encoder output for classification.

Answer: B Explanation: Autoencoders are widely used for anomaly detection. The model is trained on a large dataset of "normal" behavior. Because the autoencoder learns to reconstruct normal data well, it will have low reconstruction error for normal instances. When a new, anomalous data point is presented (e.g., a fraudulent transaction), the model will struggle to reconstruct it accurately, resulting in a high reconstruction error. This high error can then be used to flag the observation as a potential anomaly.

Q93: Sparse Autoencoder

What is a sparse autoencoder?

1. An autoencoder with fewer hidden neurons than input neurons.
2. An autoencoder where many weights (e.g., 80%) are set to zero, even with more hidden neurons than inputs.
3. An autoencoder with no hidden layers.
4. An autoencoder that uses linear activation functions.

Answer: B Explanation: A sparse autoencoder has more hidden neurons than input neurons, but many of the weights are set to zero (e.g., 80% sparsity). This forces the network to learn a sparse representation where only a small number of neurons are active for any given input. Even though there are more hidden units than features, the effective number of active units is much smaller, achieving compression while allowing for greater representational capacity.

Q94: Deep Autoencoder

What is a deep autoencoder?

1. An autoencoder with only one hidden layer.
2. An autoencoder with multiple hidden layers, typically symmetric about the bottleneck center.
3. An autoencoder that uses only linear activation functions.
4. An autoencoder used only for classification.

Answer: B Explanation: A deep autoencoder has multiple hidden layers, typically arranged symmetrically about the bottleneck (center) layer. For example, with 10 features, the architecture might be: $10 \rightarrow 5 \rightarrow 3 \rightarrow 5 \rightarrow 10$, where the center layer has 3 neurons. This deeper architecture can capture more sophisticated non-linear patterns than a single hidden layer, potentially achieving better compression and reconstruction.

Q95: Autoencoder - Linear vs Non-Linear

What happens if an autoencoder uses only linear activation functions?

1. It becomes equivalent to PCA.
2. It becomes more powerful than non-linear autoencoders.
3. It cannot be trained.
4. It becomes equivalent to K-means.

Answer: A Explanation: If an autoencoder uses only linear activation functions with a single hidden layer, and the inputs are suitably normalized, the encoder hidden nodes will be the first K principal components. This makes the linear autoencoder equivalent to PCA. However, the power of autoencoders comes from using non-linear activation functions, allowing them to capture complex, non-linear relationships and outperform PCA for many applications.

Q96: Autoencoder - Use Cases

In risk management, autoencoders are widely used for:

1. Classification of loan defaults.
2. Anomaly detection (e.g., fraudulent transactions).
3. Predicting stock prices.
4. Feature engineering.

Answer: B Explanation: In risk management, autoencoders are widely used for anomaly detection. For example, they can be trained on normal transaction patterns and then used to identify fraudulent transactions by detecting high reconstruction error. They are also used for dimensionality reduction and feature extraction, but their most common risk application is detecting unusual patterns that may indicate fraud, operational risk events, or other anomalies.

Q97: Autoencoder - Input and Output

What are the inputs and outputs of an autoencoder?

1. Inputs are labels; outputs are predictions.
2. Inputs and outputs are both the features (X); the network is trained to reconstruct its input.
3. Inputs are features; outputs are class labels.
4. Inputs are random noise; outputs are features.

Answer: B Explanation: In an autoencoder, both the inputs and outputs are the same set of features (X). The network is trained to reconstruct its own input—that is, to output $\hat{x}$ that is as close as possible to the input x. This is why autoencoders are used for unsupervised learning: no labels are required. The network learns to compress the input through the bottleneck and then reconstruct it, forcing it to learn the most important features of the data.

Q98: Autoencoder - Code

What is the "code" in an autoencoder?

1. The output layer's predictions.
2. The compressed representation (hidden layer values) of the data.
3. The input layer's raw features.
4. The reconstruction error.

Answer: B Explanation: The "code" is the compressed representation of the data at the hidden layer(s) of the autoencoder. It is the output of the encoder and the input to the decoder. The code has lower dimensionality than the original input, capturing the most important features of the data. This code can be used as a compact feature representation for other machine learning tasks, or to detect anomalies based on reconstruction error.

Q99: Autoencoder - Treasury Yield Example

In the autoencoder Treasury yield example, how did autoencoders compare to PCA?

1. Autoencoders significantly outperformed PCA.
2. PCA significantly outperformed autoencoders.
3. Autoencoders performed similarly to PCA with MSEs close to each other.
4. PCA was not applicable to the Treasury yield data.

Answer: C Explanation: In the Treasury yield example, autoencoders performed similarly to PCA, with MSEs close to each other for the same number of components/hidden units. For 1 component, autoencoder MSE was 0.18270 vs PCA 0.18208; for 2 components, 0.05027 vs 0.04965; and so on. This demonstrates that for this dataset, the linear PCA captured most of the important structure, and the non-linear autoencoder did not provide significant additional benefit.

Q100: Autoencoder - Summary

Which statement best summarizes autoencoders?

1. Autoencoders are supervised models for classification that use label information to reduce dimensionality.
2. Autoencoders are unsupervised neural networks that learn compressed representations by minimizing reconstruction error, useful for dimensionality reduction and anomaly detection.
3. Autoencoders are identical to principal component analysis in all aspects.
4. Autoencoders are only useful for image processing applications.

Answer: B Explanation: Autoencoders are unsupervised neural networks that learn compressed, low-dimensional representations of data by minimizing reconstruction error. They consist of an encoder (compressing the input) and a decoder (reconstructing the input). Their primary applications include dimensionality reduction, feature extraction, and anomaly detection. They can be non-linear (unlike PCA) and are widely used in risk management for detecting fraudulent transactions and other anomalies.